

- **Code-based cryptography:** Based on decoding random linear codes, resilient for decades.
- **Multivariate and isogeny-based schemes:** Innovative but some have already fallen to attacks.

In 2024, US-based NIST (National Institute of Standards and Technology) published the first PQC standards:

- **ML-KEM (FIPS 203):** Key encapsulation, replacing ECDH.
- **ML-DSA (FIPS 204):** Digital signatures, replacing RSA and ECDSA.
- **SLH-DSA (FIPS 205):** Hash-based signatures, ideal for long-term archival.

These are no longer research projects. They are ready for integration into TLS, SSH, S/MIME, and code signing.

Why open source must lead

Open source is the foundation of global cryptography. Migration must begin here for three reasons:

- **It builds the plumbing.** OpenSSL, GnuTLS, WireGuard, OpenSSH, and the Linux kernel supply primitives used everywhere. Without PQC support upstream, downstream projects cannot secure themselves.
- **It moves slowly.** Enterprise distributions and embedded firmware remain in production for years. PQC must land early so maintainers can test, certify, and deploy before quantum risk becomes urgent.
- **It sets the example.** When open source integrates PQC, publishes benchmarks, and runs interoperability events, it shows governments and

vendors how practical migration works. Patterns established here become industry norms.

Why migration is harder than it looks

Replacing cryptography is not a simple version bump. PQC introduces real engineering challenges:

- **Bigger payloads:** Keys and signatures are larger, affecting certificate chains, buffers, and low-bandwidth devices.
- **Protocol negotiation:** TLS, IKEv2, SSH, and Noise need agreed code points before PQC handshakes work.
- **Hybrid modes:** Many deployments will combine classical ECDH with PQC KEMs to hedge risk.
- **Testing and certification:** PQC requires unit, fuzz, and interoperability tests, plus FIPS or Common Criteria certification in regulated sectors.
- **Human factors:** Engineers must learn new names, failure modes, and configuration options without losing confidence in their systems.

A maintainer's roadmap

If your project touches cryptography, here's how to start.

Step 1: Audit your crypto surface:

Catalogue every protocol, file format, and library using RSA, ECDSA, ECDH, or DSA. Identify secrets worth protecting from 'harvest now, decrypt later'.

Step 2: Adopt trusted libraries: Do not roll your own. Use vetted upstream libraries:

- *liboqs* (Open Quantum Safe) — broad C toolkit.
- *OpenSSL 3.2+* — ML-KEM and ML-DSA via provider architecture.
- *BouncyCastle* — PQC support for Java and C#.
- Reference code at pq-crystals.org and discussions on [pqc-forum](https://pqc-forum.org).

Step 3: Enable provider

architecture: Use engines like OpenSSL 3 or Java's JCA to plug in new algorithms without rewriting applications.

Step 4: Build crypto agility: Keep algorithm identifiers, key formats, and handshake logic configurable. Avoid hard-coding key sizes or names.

Step 5: Test hybrids and

fallbacks: Ensure your system handles peers without PQC, hybrid handshakes, and downgrade attempts. Document preferred cipher suites and deprecate weak ones.

Step 6: Engage in standards

and interop: Join IETF working groups, NIST PQC forums, and Open Quantum Safe events. Interoperability testing ensures your code works across vendors.

Common pitfalls to avoid

- **Waiting for 'quantum tomorrow':** The real risk is traffic already being harvested.
- **Rolling your own primitives:** PQC math is subtle; use vetted libraries.
- **Ignoring performance:** Larger payloads affect latency and bandwidth. Test early.
- **Hard-coding algorithms:** Build crypto agility into configuration.
- **Forgetting training:** Migration

PQC standards at a glance

Standard	Purpose	Replaces	Status	Recommended use case
ML-KEM	Key encapsulation	ECDH	FIPS 203 (2024)	TLS, VPNs
ML-DSA	Digital signatures	RSA, ECDSA	FIPS 204 (2024)	Certificates, code signing
SLH-DSA	Hash-based signatures	RSA, ECDSA	FIPS 205 (2024)	Archival, long-term signatures
SHA-3 / SHAKE	Hashing/XOFs	SHA-2 (in PQC modes)	Standardised	PQC-compatible hashing

■ ■ Continue to page...72